



Fix the Turtle Code

Quick facilitation notes & answer key

Goal

Students debug turtle loops (C3.2): decide what the code should draw, then find and fix the wrong range count or turn angle. The challenge separates resizing (change forward) from changing the number of sides (range).

Curriculum (Ontario Math 2020, Strand C3)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential, concurrent, and repeating events.

C3.2 — read and alter existing code, including code that involves sequential, concurrent, and repeating events, and describe how changes to the code affect the outcomes.

Materials

Python Turtle projected (or a web playground) to fix bugs live; the printed worksheet + a pencil per student.

Run it (about 30 min)

1. Run the buggy `range(3)` square; the open shape makes the bug visible.
2. Ask 'what should it draw?' then find the wrong number together; fix and re-run.
3. Students do Exercises 1–4 (Ex 4 is the stretch), solo or in pairs.
4. Share answers; land the big idea: `range` = sides, `turn` = corner, `forward` = size.

Answer key (checked by the run-gate)

Ex 1 — `range(4)`.

Ex 2 — `right(120)`.

Ex 3 — `range(5)` ($5 \times 20 = 100$).

Ex 4 (challenge) — change forward to a bigger number (keep `range(4)`). `range(8)` just retraces the same square; `forward` sets the size.

Watch for & differentiate

Common slip: changing the range to resize (Ex 4) instead of changing forward.

Support: write the GOAL (square / triangle / line) before hunting the bug.

Extend: ask the fix for a pentagon drawn with `right(90)` (use `right(72)`).

Success looks like

The student names what the code should draw, fixes the wrong range or turn (Ex 1–3), and explains that resizing means changing forward, not the range (Ex 4).